

10-Week ML/AI Internship Prep: Plan Evaluation & Complete Roadmap

TL;DR

Your original study guide is a **solid 7.5/10** — well-structured and covers the right topics, but it critically misses **Data Structures & Algorithms (DSA)** and underestimates the time needed for hands-on projects. For an ML/AI internship, DSA isn't just “nice to have” — it's the gatekeeper to getting past technical interviews. With 40 hours/week over 10 weeks, this enhanced plan adds DSA alongside your original roadmap, integrates the best free resources for each topic, and allocates **20+ hours in the final weeks exclusively to portfolio projects**. The result: a rigorous, interview-ready preparation that transforms your guide from a learning curriculum into a job-winning system.

1. Rating Your Original Plan: Strengths & Gaps

Your original study guide demonstrates excellent strategic thinking about ML/AI skill acquisition. The phased approach — moving from Python foundations through data literacy, hands-on AI models, core ML, LLMs, and finally real-world projects — follows a pedagogically sound progression. As someone with a CE background transitioning to MSCS, you're in an ideal position to execute this because you already understand programming fundamentals, computer architecture, and likely have some exposure to algorithms. This isn't starting from zero; it's redirecting existing engineering instincts toward ML/AI-specific tooling and workflows.

The plan's most significant strength is its **emphasis on project-based learning**. Too many ML learners get stuck in “tutorial hell” — consuming courses without ever building anything original. Your Step 6 (“Build Lots of Real-World AI Applications”) and the inclusion of mini-project workflows within each step show you understand that **portfolios get interviews, not certificates**. The coverage of modern AI tooling — LangChain, RAG, vector databases, Streamlit — is particularly strong and aligned with what ML engineering teams actually use in 2025 ([wordpress.com](https://www.wordpress.com)) . The recognition that no-code/low-code tools (Crew AI, Langflow, n8n) have a place in an engineer's toolkit also shows practical maturity — knowing when to reach for a visual workflow builder versus writing Python from scratch is a genuinely valuable skill.

However, there are **three critical gaps** that would prevent this plan from reliably landing an ML/AI internship. First and most importantly: **the complete absence of Data Structures & Algorithms (DSA)**. Technical interviews for ML/AI roles still test coding problem-solving — not at FAANG-hard levels necessarily, but you'll absolutely face LeetCode-style questions. LogicMojo's analysis of 200+ successful AI placements found that candidates who practiced DSA alongside ML learning had significantly higher success rates ([logicmojo](https://logicmojo.com)) . Second, the original plan lacks specific resource recommendations beyond generic library names — “learn pandas” is good direction, but without knowing whether to use Kaggle Learn, DataCamp, or a YouTube playlist, a beginner wastes hours evaluating options instead of learning. Third, the timeline is too compressed for genuine competency — 10 weeks is achievable, but only with extremely focused resource selection and aggressive project milestones. Without these additions, the plan risks producing a learner who can discuss ML concepts but struggles to pass a coding screen or demonstrate production-ready skills.

Aspect	Original Plan	Enhanced Plan (This Guide)
DSA Coverage	None —critical gap	8-12 hrs/week dedicated LeetCode practice
Resource Specificity	Generic library names only	Curated courses, books, tutorials with links
Project Milestones	Vague “build projects” step	4 portfolio projects with weekly deliverables
Interview Prep	Not addressed	Integrated DSA + system design fundamentals
Time Allocation	Unclear	40 hrs/week with precise hourly breakdowns
LLM/AI Agents Depth	Good coverage	Enhanced with LangChain v0.3 + Ollama local LLMs
Real-World Deployment	Streamlit mentioned	Flask APIs, Docker, GitHub Actions added

2. Why DSA Is Non-Negotiable for ML/AI Internships

The ML/AI field has a peculiar dual gatekeeping system. On one side, you need ML competency — knowing how to clean data, train models, evaluate performance, and deploy solutions. On the other side, you need **software engineering fundamentals** — and DSA is the universal language of technical interviewing. Even ML research internships at top companies include coding assessments. The NeetCode 150/250 curriculum, built by a former Google SWE, has become the de facto standard for structured DSA preparation because it organizes problems by topic with dependency-aware sequencing ([PracHub](#)). Following this alongside your ML learning ensures you don’t become the candidate who can explain transformer architectures but can’t invert a binary tree.

The good news is that **DSA for ML roles is less intense than for pure SWE roles**. You’re not grinding LeetCode Hard graph problems daily. Most ML/AI internship interviews focus on Arrays, HashMaps, Two Pointers, Sliding Window, basic Trees, and simple Dynamic Programming — roughly the first half of NeetCode 150 ([logicmojo](#)). The recommended approach in this plan dedicates **8-12 hours per week consistently** to DSA practice, which is sufficient to build interview-ready competency without derailing your ML learning. This parallel-track approach mirrors what successful candidates actually do — LogicMojo’s mentorship data shows that students who ran DSA practice alongside AI learning (rather than sequencing them) had measurably better placement outcomes ([logicmojo](#)).

DSA practice also has a surprising secondary benefit for ML work: **it sharpens the exact type of structured thinking that debugging ML pipelines requires**. When you’re tracking down why your neural network’s loss isn’t converging, the systematic debugging approach you developed solving LeetCode problems transfers directly. The pattern-matching skills from recognizing that a problem is a “sliding window” variant mirror the intuition for recognizing that a dataset needs a particular preprocessing approach. DSA isn’t interview trivia — it’s deliberate practice for the problem-solving loops you’ll run constantly as an ML engineer.

3. The 10-Week Schedule: Complete Breakdown

The schedule below allocates **40 hours per week** across six focus areas: Python (CS50P), DSA (Neet-Code/LeetCode), SQL & Data Literacy, ML Fundamentals (scikit-learn/PyTorch), AI/LLMs & Agents (LangChain/RAG), and Projects & Portfolio. The allocation shifts progressively from foundations (Weeks 1-3) through ML core (Weeks 4-6) and AI/LLMs (Weeks 7-8) to an intensive project sprint (Weeks 9-10).

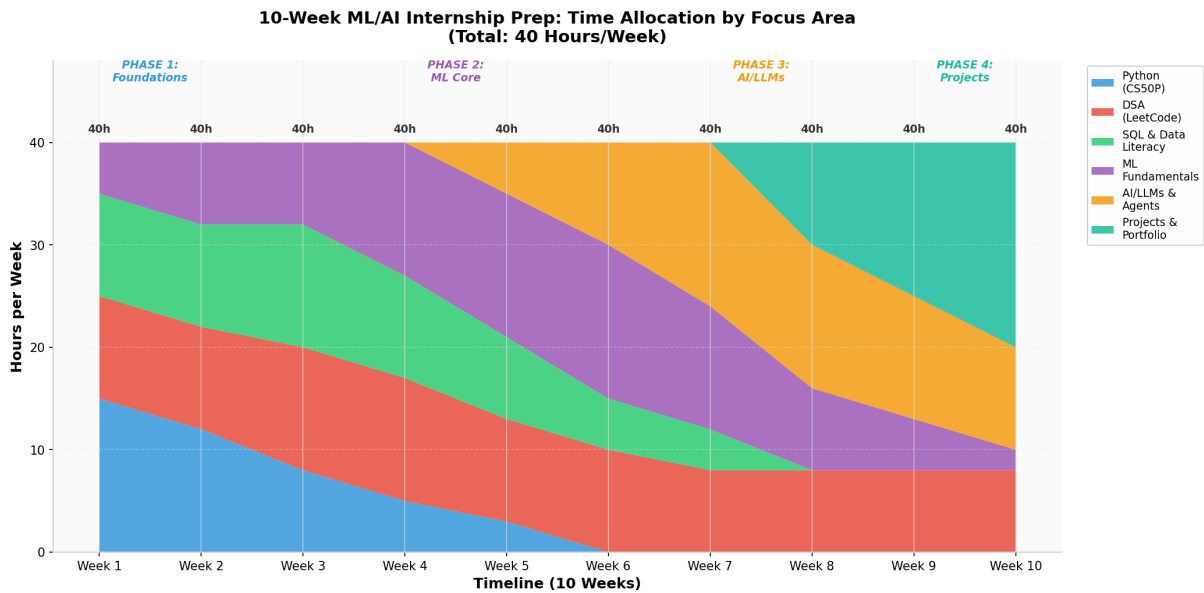


Figure 1: 10-Week Schedule Timeline

4. Phase 1: Foundations (Weeks 1-3)

Week 1: Python Deep Dive + DSA Kickoff

Day	Focus Area	Activity	Hours	Resource
1-3	Python	CS50P Lectures 0-3 (Functions, Variables, Conditionals)	5	CS50P
4-5	Python	CS50P Problem Sets 0-3	4	CS50P Problem Sets
6-7	Python	CS50P Lectures 4-5 (Libraries, Unit Tests)	4	CS50P
Daily	DSA	NeetCode Arrays & Hashing (5 problems)	10	NeetCode 150
Daily	SQL	SQLBolt Lessons 1-9	10	SQLBolt
5-7	Python	Personal scripting project (web scraper or automation)	7	Self-directed

Total: 40 hours | Python: 15h, DSA: 10h, SQL: 10h, ML: 5h

Week 1 front-loads Python because your CE background means you can accelerate through CS50P faster than a complete beginner. CS50P is the best Python foundation course available — it’s taught by David Malan with Harvard-level rigor, covers not just syntax but code style, type hints, docstrings, list comprehensions, and generators that most beginner courses skip ([rivenintech.com](https://www.rivenintech.com)) . The problem sets are genuinely engaging and build real skills. Don’t skip the “Et Cetera” lecture — it covers Pythonic patterns you’ll use constantly.

DSA starts immediately with NeetCode’s **Arrays & Hashing** section because these are the most frequently asked problem types and they require minimal algorithmic theory to get started. The pattern of “use a HashMap for O(1) lookups” is something you’ll internalize quickly and use constantly. Aim for **5 problems in the first week** — quality over quantity, with full understanding of the optimal solution.

SQLBolt is the fastest path to SQL fundamentals — it’s entirely browser-based, requires no setup, and covers 90% of the SQL you’ll write in real-world data work ([ai2sql.io](https://sqlbolt.io)) . Lessons 1-9 take you from basic SELECT queries through JOINS and NULL handling. The interactive exercises give immediate feedback, which is crucial for early SQL learning where syntax errors are frustratingly opaque.

Week 2: Python Mastery + DSA Progression

Day	Focus Area	Activity	Hours	Resource
1-3	Python	CS50P Lectures 6-8 (File I/O, Regular Expressions, OOP)	5	CS50P
4-5	Python	CS50P Final Project + Problem Sets	5	CS50P
6-7	Python	Kaggle Python micro-course + Pandas intro	4	Kaggle Learn
Daily	DSA	NeetCode Two Pointers + Stack (5 problems)	10	NeetCode 150
Daily	SQL	SQLBolt Lessons 10-18 + Kaggle SQL course start	10	SQLBolt + Kaggle
5-7	Data	NumPy fundamentals (array operations, broadcasting)	6	NumPy Fundamentals (Datacamp)

Total: 40 hours | Python: 12h, DSA: 10h, SQL: 10h, ML: 8h

Week 2 completes CS50P and transitions into data-specific Python tooling. The Kaggle Python micro-course is excellent for data science-oriented Python — it’s concise, practical, and browser-based ([sub-stack.com](https://www.kaggle.com/learn/python)) . Immediately follow it with Kaggle’s Pandas micro-course since pandas is the primary data manipulation tool you’ll use for all subsequent ML work. The Kaggle ecosystem is particularly valuable because all exercises run in-browser with real datasets, eliminating setup friction.

DSA progression moves into **Two Pointers and Stack** problems. These patterns are less common in ML interviews than Arrays/Hashing but build crucial algorithmic intuition. The Two Pointer technique (maintaining left/right pointers that converge) appears in problems involving sorted arrays and palindromes — the pattern recognition skills transfer to optimizing data processing pipelines.

SQL learning expands from SQLBolt's basics into Kaggle's SQL micro-course, which introduces window functions and more complex aggregations that SQLBolt intentionally skips (ai2sql.io) . Window functions (ROW_NUMBER, RANK, LAG, LEAD) are essential for time-series analysis and feature engineering in ML pipelines — they're the difference between basic SQL fluency and data-engineering-ready skills.

Week 3: Data Literacy + DSA Consolidation

Day	Focus Area	Activity	Hours	Resource
1-3	Data	Pandas deep dive: merging, groupby, time-series	6	Kaggle Learn + Pandas docs
4-5	Data	Matplotlib & Seaborn visualization mastery	6	Seaborn Tutorials (KDnuggets) (KDnuggets)
6-7	Data	End-to-end EDA project on Kaggle dataset	6	Self-directed
Daily	DSA	NeetCode Binary Search + Sliding Window (5 problems)	12	NeetCode 150
Daily	SQL	SQL practice on DataLemur/LeetCode SQL (10 problems)	10	DataLemur

Total: 40 hours | DSA: 12h, SQL: 10h, Data: 12h, ML: 6h

Week 3 is the **data literacy consolidation week**. Pandas merging, multi-indexing, and time-series handling are the operations you'll perform in virtually every ML project. The Kaggle Learn micro-courses combined with the official pandas documentation provide both guided learning and reference material. For visualization, seaborn's statistical plotting capabilities (distribution plots, regression plots, categorical plots) produce publication-quality figures with minimal code — understanding when to use each plot type is a skill that distinguishes competent data scientists from beginners ([KDnuggets](#)) .

The EDA (Exploratory Data Analysis) project is your first mini-portfolio piece. Pick a Kaggle dataset with interesting features — the Titanic dataset is classic for beginners, or choose something domain-relevant to your interests. The goal isn't model building yet; it's demonstrating that you can load data, handle missing values, create informative visualizations, and extract actionable insights. This project goes on your GitHub with a clean README.

DSA progresses to **Binary Search and Sliding Window** — both are high-frequency interview patterns. Binary Search variants (finding boundaries, rotated arrays) appear constantly because they're easy to explain but have subtle edge cases that trip up unprepared candidates. Sliding Window is essential for substring/subarray problems and has direct analogies to batch processing in ML pipelines.

5. Phase 2: Machine Learning Core (Weeks 4-6)

Week 4: ML Fundamentals Begin

Day	Focus Area	Activity	Hours	Resource
1-3	ML	Andrew Ng ML Specialization Course 1 (Supervised Learning)	8	Coursera: ML Specialization (wordpress.com)
4-5	ML	Hands-On ML Book Ch. 1-2 (End-to-end ML project)	8	Hands-On ML (Géron) (ML in Production)
6-7	ML	scikit-learn: Linear/Logistic Regression implementation	5	scikit-learn docs
Daily	DSA	NeetCode Tree problems (BFS/DFS)	12	NeetCode 150
5-7	SQL	Advanced SQL: window functions practice	7	LeetCode SQL + Mode Analytics

Total: 40 hours | ML: 13h, DSA: 12h, SQL: 10h, Python: 5h

Week 4 introduces the **Andrew Ng Machine Learning Specialization** — the single most recommended ML course for beginners, with over 5 million enrolled students ([wordpress.com](#)) . The specialization’s strength is diagnostic vocabulary: understanding why models underfit, overfit, or fail to generalize. This conceptual foundation prevents the common beginner mistake of throwing neural networks at every problem without understanding whether the issue is data quality, feature engineering, or model selection. The course uses Python, NumPy, and scikit-learn — tools you’re already familiar with from Weeks 1-3.

Parallel to the course, start reading **Aurélien Géron’s “Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow”** — widely considered the best practical ML book ([FreeCodecamp](#)) . The book’s Chapter 2 presents a complete end-to-end ML project (predicting California housing prices) that demonstrates the full workflow: data exploration, preprocessing, model selection, cross-validation, hyperparameter tuning, and deployment. Working through this alongside Andrew Ng’s theory creates a powerful learn-by-doing loop. Géron’s book is particularly valuable because it’s written by a practitioner (former YouTube video classification lead) and includes production-ready techniques that courses often skip ([ML in Production](#)) .

scikit-learn is the primary library for classical ML — it implements virtually every algorithm you’ll need with a consistent API: `fit()`, `predict()`, `score()` ([Wikitechy](#)) . By Week 4, you should be comfortable with the standard workflow: `train_test_split` → preprocessing pipeline → model fitting → evaluation metrics. Implementing Linear Regression and Logistic Regression from scratch (using NumPy) alongside scikit-learn’s built-in versions deepens understanding of what the library is actually doing under the hood.

Week 5: ML Algorithms + First ML Project

Day	Focus Area	Activity	Hours	Resource
1-3	ML	Classification algorithms: SVM, Decision Trees, Random Forest	6	Hands-On ML Ch. 5-7

Table 6 – continued

Day	Focus Area	Activity	Hours	Resource
4-5	ML	Unsupervised learning: K-Means, PCA	4	Andrew Ng Course 3 + Hands-On ML
6-7	ML	Project: Customer Churn Predictor	6	Self-directed
Daily	DSA	NeetCode Heap + Backtracking (5 problems)	10	NeetCode 150
5-7	AI/LLMs	OpenAI API basics: completions, embeddings	5	OpenAI API docs
3-5	Data	Feature engineering techniques	9	Hands-On ML Ch. 2 + Kaggle notebooks

Total: 40 hours | ML: 14h, DSA: 10h, AI: 5h, Data: 9h, Projects: 2h

Week 5 expands your algorithm toolkit while starting your **first significant ML project: a Customer Churn Predictor**. This is a classic binary classification problem that demonstrates end-to-end ML competency — data collection (use a public telecom dataset), cleaning, feature engineering, model training (compare Logistic Regression, Random Forest, and XGBoost), evaluation (precision, recall, F1-score — not just accuracy), and a simple Streamlit dashboard for predictions. The project should be GitHub-ready with a comprehensive README explaining your methodology and results.

Feature engineering is where ML transitions from “running library functions” to “genuine data science intuition”. Understanding when to create interaction terms, how to handle categorical variables (one-hot encoding vs. target encoding), and why feature scaling matters for some algorithms but not others — these decisions separate pipeline operators from ML practitioners. Hands-On ML Chapter 2 covers these techniques comprehensively ([ML in Production](#)) .

The first exposure to **OpenAI’s API** marks the transition toward AI engineering. Understanding how to structure prompts, handle API responses, and manage rate limits is foundational for all subsequent LLM work. Start with simple completion requests before moving to embeddings — the embeddings API is what powers RAG systems, which you’ll build in Week 8.

Week 6: Deep Learning Introduction + Neural Networks

Day	Focus Area	Activity	Hours	Resource
1-3	ML	Neural Networks: feed-forward, backpropagation	8	Andrew Ng Course 2 + Hands-On ML Ch. 10
4-5	ML	PyTorch fundamentals (tensors, autograd, nn.Module)	8	PyTorch Official Tutorials (mljourney.com)
6-7	ML	Build first neural network: MNIST classifier	5	PyTorch tutorials

Table 7 – continued

Day	Focus Area	Activity	Hours	Resource
Daily	DSA	NeetCode Graph + Dynamic Programming basics	10	NeetCode 150
5-7	AI/LLMs	Ollama setup + local LLM experimentation	10	Ollama Tutorial (T Recommendation H.263)

Total: 40 hours | ML: 15h, DSA: 10h, AI: 10h, Projects: 5h

Week 6 introduces **deep learning with PyTorch** — and this is where framework choice matters. Research consistently shows PyTorch is the better choice for beginners in 2025: it's more Pythonic, has cleaner debugging (standard Python debuggers work), and dominates the research community which means better tutorials and pre-trained models ([acecloud.ai](#)). TensorFlow's Keras API is simpler for basic models, but PyTorch's flexibility pays dividends when you move to custom architectures. For ML internships, PyTorch knowledge is increasingly expected — especially for roles involving LLMs or computer vision.

Understanding neural networks at the implementation level (not just calling `model.fit()`) is crucial. Building MNIST from scratch with PyTorch — defining the `nn.Module`, implementing the training loop with `autograd`, and manually computing loss — gives you the intuition that makes debugging complex models possible later. The jump from scikit-learn's declarative API to PyTorch's imperative style is significant but necessary for any serious ML work.

Ollama becomes your local LLM infrastructure this week. Setting up Ollama to run models like Llama 3.1 locally gives you unlimited experimentation without API costs ([T Recommendation H.263](#)). The setup is remarkably simple — one-line installation, then `ollama run llama3.1` gets you a ChatGPT-like interface running entirely on your machine. This enables rapid experimentation with prompt engineering, model comparison, and eventually RAG pipeline development without worrying about per-token pricing.

6. Phase 3: AI, LLMs & Agents (Weeks 7-8)

Week 7: LangChain, RAG & Vector Databases

Day	Focus Area	Activity	Hours	Resource
1-2	AI/LLMs	LangChain fundamentals: Chains, Prompts, Models	6	LangChain Crash Course (Udemy) (Udemy)
3-4	AI/LLMs	Vector databases: ChromaDB setup & embeddings	6	ChromaDB docs (Agix Technologies)
5-6	AI/LLMs	Build RAG pipeline: document ingestion → retrieval → generation	8	LangChain RAG Tutorial

Table 8 – continued

Day	Focus Area	Activity	Hours	Resource
7	AI/LLMs	RAG evaluation & optimization	4	Self-directed
Daily	DSA	LeetCode Mixed practice (previously covered topics)	8	LeetCode
5-7	ML	Model evaluation & hyperparameter tuning deep dive	8	Hands-On ML Ch. 2 + scikit-learn docs

Total: 40 hours | AI/LLMs: 16h, DSA: 8h, ML: 12h, Projects: 4h

Week 7 is where your plan’s **AI/LLM focus truly materializes**. LangChain has become the standard framework for building LLM applications, and the latest v0.3 release significantly streamlined the API ([Udemy](#)) . The core abstractions — `ChatModels`, `PromptTemplates`, `Chains`, and `Agents` — provide a consistent interface for combining LLMs with external data sources and tools. Understanding when to use a simple Chain versus a full Agent with tool-calling capabilities is a key architectural decision in LLM application design.

Retrieval-Augmented Generation (RAG) is the most important LLM pattern for practical applications. Instead of relying on an LLM’s static training knowledge, RAG retrieves relevant documents from a vector database and injects them into the prompt context — dramatically improving accuracy for domain-specific queries ([wordpress.com](#)) . Building a complete RAG pipeline involves: (1) document loading and chunking, (2) embedding generation (using OpenAI or local models via Ollama), (3) vector storage in ChromaDB, (4) similarity search for relevant chunks, and (5) response generation with retrieved context. This is a substantial project that becomes a major portfolio piece.

ChromaDB is the recommended vector database for beginners — it’s lightweight, open-source, and has excellent LangChain integration ([Agix Technologies](#)) . For prototyping and learning, it requires zero infrastructure setup (runs embedded in your Python process). Understanding vector similarity search — how embeddings convert text into high-dimensional vectors and how cosine similarity finds “close” vectors — is foundational modern AI engineering knowledge.

Week 8: AI Agents + Project Integration

Day	Focus Area	Activity	Hours	Resource
1-2	AI/LLMs	LangGraph: building multi-agent workflows	6	LangGraph docs
3-4	AI/LLMs	Build AI Email Triage Bot (LangChain + RAG + Streamlit)	10	Self-directed
5	AI/LLMs	No-code tools: explore Langflow, Crew AI	4	Langflow
6-7	Projects	Deploy projects: Flask APIs, Docker basics	6	Flask docs + Docker docs

Table 9 – continued

Day	Focus Area	Activity	Hours	Resource
Daily	DSA	LeetCode mixed + SQL practice	8	LeetCode + DataLemur
7	Career	GitHub cleanup, resume draft, LinkedIn update	6	Self-directed

Total: 40 hours | AI/LLMs: 14h, Projects: 10h, DSA: 8h, Career: 6h, ML: 2h

Week 8 integrates everything into **production-like projects**. The AI Email Triage Bot combines multiple skills: using LangChain to process emails, vector storage for historical context, LLM summarization, and a Streamlit dashboard for user interaction. This isn't a toy project — it's architecturally similar to real enterprise AI tools. Building it demonstrates that you can combine APIs, databases, and UI components into a cohesive application.

LangGraph (LangChain's extension for building agent workflows) enables multi-step reasoning where an agent can plan, execute tools, observe results, and iterate — the “agent loop” pattern that powers sophisticated AI applications (onlinetutorials.me). Understanding this pattern is increasingly important as companies move beyond simple Q&A chatbots to AI systems that can actually perform tasks.

Deployment skills — building Flask APIs, containerizing with Docker, and hosting on platforms like Render or Hugging Face Spaces — transform notebooks into production artifacts. An ML model that only runs on your laptop is a prototype; an ML model accessible via REST API is a product. For internships, demonstrating deployment awareness signals that you understand the full ML lifecycle, not just the training phase.

7. Phase 4: Project Sprint & Interview Prep (Weeks 9-10)

Week 9: Portfolio Project Development

Day	Focus Area	Activity	Hours	Resource
1-3	Projects	Project: AI Knowledge Base Assistant (full RAG app)	15	Self-directed
4-5	Projects	Project: Data-Driven Recommendation Engine	10	Self-directed
6-7	Projects	Polish GitHub repos, write READMEs, create demos	7	Self-directed
Daily	DSA	LeetCode medium problems + mock interview practice	8	LeetCode + Pramp

Total: 40 hours | Projects: 25h, DSA: 8h, Polish: 7h

Week 9 is **project sprint week**. The goal is completing two substantial portfolio projects:

Project 1: AI Knowledge Base Assistant — A full RAG application that ingests documents (PDFs, Markdown files, or web pages), stores them in a vector database, and answers questions via a Streamlit interface. This demonstrates LangChain, vector databases, embeddings, and UI development. Deploy it on Streamlit Community Cloud for a live demo link you can share with recruiters.

Project 2: Data-Driven Recommendation Engine — Use clustering (K-Means or DBSCAN) on a purchase history dataset to segment users, then build a simple content-based recommender using scikit-learn. Expose recommendations via a Flask API with proper documentation. This demonstrates classical ML skills alongside API development — showing versatility across the ML stack.

The key to portfolio projects is **README quality**. Each project needs: (1) clear problem statement, (2) tech stack list, (3) setup instructions, (4) architecture diagram, (5) demo screenshots or GIFs, and (6) discussion of challenges faced and trade-offs made. A project with a great README gets attention; a project with code-only gets ignored.

Week 10: Final Polish + Interview Readiness

Day	Focus Area	Activity	Hours	Resource
1-3	Projects	Capstone: End-to-End ML Pipeline (MLOps-style)	15	Self-directed
4-5	Projects	Deploy all projects, create portfolio website	8	GitHub Pages + Streamlit
6-7	DSA	Intensive LeetCode: focus on top 50 most common	10	LeetCode + NeetCode
5-7	Career	Resume finalization, LinkedIn optimization, apply to internships	7	Self-directed

Total: 40 hours | Projects: 23h, DSA: 10h, Career: 7h

Week 10's **capstone project** ties everything together: an end-to-end ML pipeline that includes data ingestion, preprocessing, model training with experiment tracking (MLflow), automated evaluation, and API deployment. This demonstrates MLOps awareness — the engineering practices that separate academic ML from production ML. Even a simplified version (using GitHub Actions for automation instead of Airflow) shows you understand that ML systems require ongoing maintenance, not just one-time training.

Final DSA push focuses on the most commonly asked problem types. By Week 10, you should have completed roughly **60-80 NeetCode problems** across Arrays, Hashing, Two Pointers, Sliding Window, Binary Search, Trees, and basic Graph/DP. This is sufficient for ML/AI internship interviews, which typically don't reach the difficulty of pure SWE roles ([logicmojo](#)). Practice explaining your solutions out loud — communication is as important as correctness in interviews.

The final career tasks — resume polish, LinkedIn optimization, and beginning applications — should happen simultaneously with project completion. Your resume should lead with technical skills (Python,

PyTorch, scikit-learn, LangChain, SQL) followed by project descriptions with quantified outcomes (“Built RAG pipeline achieving 85% answer relevance on domain-specific queries”). Apply broadly: LinkedIn, company career pages, AngelList for startups, and your university’s career services. The goal in Week 10 isn’t to get an offer — it’s to get conversations started while your skills are fresh.

8. Curated Resource Guide: The Best Free Tools for Each Topic

Python Foundations

Resource	Type	Time	Best For
CS50P (Harvard) (rivenintech.com)	Video course + problem sets	30-40 hours	Best Python foundation for engineers
Kaggle Learn Python (substack.com)	Interactive micro-course	6-10 hours	Data-science-oriented Python basics
Real Python	Articles + tutorials	Ongoing	Reference for specific topics

CS50P is the undisputed best starting point. David Malan’s teaching style, the engaging problem sets, and the coverage of Pythonic patterns (comprehensions, generators, type hints) make it superior to generic Python courses ([rivenintech.com](https://cs50p.harvard.edu/)). The course assumes no prior programming experience, but your CE background means you’ll move through the early lectures quickly and benefit most from the later lectures on libraries, file I/O, and code style.

Data Structures & Algorithms

Resource	Type	Time	Best For
NeetCode 150 (PracHub)	Curated problem list + videos	80-100 hours	Structured DSA interview prep
LeetCode	Problem platform	Ongoing	Practice problems, mock assessments
DataLemur	SQL + DSA problems	10-20 hours	Data science interview focus

NeetCode’s structured roadmap is the highest-ROI DSA resource for time-constrained learners. The dependency-aware sequencing (master Arrays before Two Pointers, learn Trees before Graphs) eliminates the random-walk problem of unsorted LeetCode practice ([PracHub](https://neetcode.io/)). For ML/AI roles, focus on the first 100 problems — the remaining 50 are more relevant for FAANG SWE roles than ML internships.

SQL & Data Literacy

Resource	Type	Time	Best For
SQLBolt (ai2sql.io)	Interactive browser tutorial	4-8 hours	Zero-setup SQL fundamentals
Kaggle Learn SQL	Interactive micro-course	6-8 hours	Data science SQL patterns

Table 14 – continued

Resource	Type	Time	Best For
Mode Analytics SQL	Tutorial + real datasets	10-15 hours	Analytics-focused SQL
DataLemur	Practice problems	5-10 hours	SQL interview preparation

SQLBolt is the fastest on-ramp — no signup, no installation, immediate feedback (ai2sql.io) . But it intentionally stops at basics, so follow it with Kaggle’s SQL course for window functions and complex joins (mimo.org) . Mode Analytics has the best free tutorial for analytics-style SQL (GROUP BY, window functions, CTEs) with realistic datasets. For ML specifically, focus on: SELECT/JOIN fluency, aggregation with GROUP BY, window functions for time-series, and subqueries for feature engineering.

Machine Learning Fundamentals

Resource	Type	Time	Best For
Andrew Ng ML Specialization (wordpress.com)	Coursera course (3 courses)	40-60 hours	ML theory + conceptual foundation
Hands-On ML (Géron) (ML in Production)	Book (O’ Reilly)	60-80 hours	Practical ML implementation
scikit-learn docs + examples	Documentation	Ongoing	Reference for specific algorithms
Kaggle Competitions	Competitions + notebooks	Ongoing	Real-world problem practice

Andrew Ng gives you the map; Géron’s book gives you the workshop ([CourseFacts](https://coursefacts.com)) . This pairing is the most effective ML learning combination available. The specialization’s strength is diagnostic vocabulary — understanding *why* models fail — while the book provides production-ready code patterns ([ML in Production](https://mlinproduction.com)) . For a 10-week timeline, focus on: Courses 1-2 of the specialization (Supervised Learning + Neural Networks) and Chapters 1-7, 10-11 of Hands-On ML. Save the rest for post-internship deepening.

Deep Learning Framework

Resource	Type	Time	Best For
PyTorch Official Tutorials (mljourney.com)	Code tutorials	15-20 hours	Deep learning implementation
Fast.ai (Practical Deep Learning) (CourseFacts)	Free course	40-50 hours	Top-down deep learning
Deeplizard PyTorch Playlist (FreeCodecamp)	YouTube videos	10-15 hours	Visual learners

PyTorch is the recommended framework for beginners in 2025. It’s more Pythonic than TensorFlow, dominates research (meaning better pre-trained models and community support), and has cleaner debugging (acecloud.ai) . TensorFlow’s Keras API is simpler for basic models, but PyTorch’s flexibility

pays dividends for any custom architecture work. The official PyTorch tutorials — especially the “Learn the Basics” series and the MNIST classifier walkthrough — provide the most reliable starting point.

AI/LLMs & Agents

Resource	Type	Time	Best For
LangChain Crash Course (Udemy) (Udemy)	Video course	6-8 hours	LangChain v0.3 fundamentals
LangChain RAG Tutorial (LangChain)	Official docs	4-6 hours	Building RAG pipelines
Ollama + local LLMs (T Recommendation H.263)	Tool + practice	4-6 hours	Local LLM experimentation
ChromaDB docs (Agix Technologies)	Documentation	2-3 hours	Vector database basics
LangGraph docs	Documentation	3-4 hours	Multi-agent workflows

The LangChain Crash Course for Beginners 2025 is the most up-to-date resource for the current v0.3 API (Udemy) . LangChain evolves quickly, and older tutorials often use deprecated APIs that won’t work with current versions. For RAG specifically, the official LangChain RAG tutorial provides a complete working example that you can adapt for your projects (LangChain) . Ollama is essential infrastructure — running models locally eliminates API costs and enables rapid iteration (T Recommendation H.263) .

Deployment & Tools

Resource	Type	Time	Best For
Streamlit docs	Documentation	3-4 hours	ML app UIs
Flask Quickstart	Documentation	2-3 hours	REST APIs for ML models
Docker for ML (tutorial)	Tutorial	3-4 hours	Containerization
Hugging Face Spaces	Platform	1-2 hours	Free ML app hosting

9. Expected Outcomes: What You’ll Know by Week 10

By the end of this 10-week program, you will have developed **demonstrable competency** across the full ML/AI stack — not just theoretical knowledge, but implemented, deployed projects that prove your skills to interviewers. The portfolio you’ll build consists of 4-5 substantial projects: an EDA analysis, a classical ML model (churn prediction), a neural network classifier, a RAG-based AI assistant, and an end-to-end ML pipeline. Each project lives on GitHub with professional documentation, and at least two will have live deployments you can demo during interviews.

Your DSA preparation will position you to confidently handle **LeetCode Medium-level problems** in Arrays, Hashing, Two Pointers, Sliding Window, Binary Search, Trees, and basic Graph traversal. For ML/AI internships, this is the standard technical bar — you’re not competing with SWE candidates who grind Hard problems, but you need to demonstrate clean, efficient code under interview conditions (logicmojo) . The consistent 8-12 hours/week of DSA practice builds both the problem-solving skills and the interview stamina (solving problems while explaining your thinking) that technical screens require.

Perhaps most importantly, you'll have developed **engineering judgment** — the ability to choose the right tool for a given problem. You'll understand when scikit-learn is sufficient versus when PyTorch is necessary, when a simple API call beats a custom model, when vector databases add value versus when they're overkill, and when no-code tools accelerate delivery versus when they create technical debt. This judgment is what distinguishes engineers from tool-users, and it's what hiring managers look for in internship candidates who might convert to full-time roles.

Competency Area	Week 1 Level	Week 10 Level
Python	Basic syntax	Fluent with pandas, NumPy, scikit-learn, PyTorch
DSA	None	60-80 problems solved, interview-ready
SQL	None	Complex queries, window functions, joins
ML Theory	None	Understands supervised/unsupervised, overfitting, regularization
ML Implementation	None	End-to-end pipelines with scikit-learn + PyTorch
LLMs/AI	None	RAG systems, LangChain agents, local LLM deployment
Deployment	None	Flask APIs, Streamlit UIs, Docker containers
Portfolio	Empty	4-5 projects on GitHub with live demos

10. Final Recommendations & Success Factors

The single most important success factor for this plan is **consistency over intensity**. Forty hours per week is substantial but sustainable if you maintain daily DSA practice (1.5-2 hours) and block larger chunks for project work. Missing a day of DSA practice creates a compounding deficit — the pattern recognition that makes you fast at problem-solving requires repeated exposure. Conversely, the ML/AI content is more forgiving of scheduling variations because projects can absorb focused weekend sessions.

Start applying before you feel “ready”. The job market rewards momentum — early applications generate interview practice (which reveals gaps you can fill), feedback on your resume, and potentially early offers that reduce pressure. Begin submitting applications in Week 8, even as you're still building projects. Many companies have multi-week interview processes, so applications submitted in Week 8 might lead to interviews in Week 10 when your projects are polished.

Finally, **document your learning publicly**. Write short LinkedIn posts about what you're building, share GitHub repos with descriptive READMEs, and consider writing 1-2 Medium articles explaining a project or a concept you learned deeply. Public learning signals enthusiasm, creates opportunities for mentorship connections, and makes you discoverable to recruiters who search for candidates by project keywords. The ML/AI community values openness — contributing what you learn back to the community, even as a beginner, builds relationships that lead to referrals and insider opportunities.